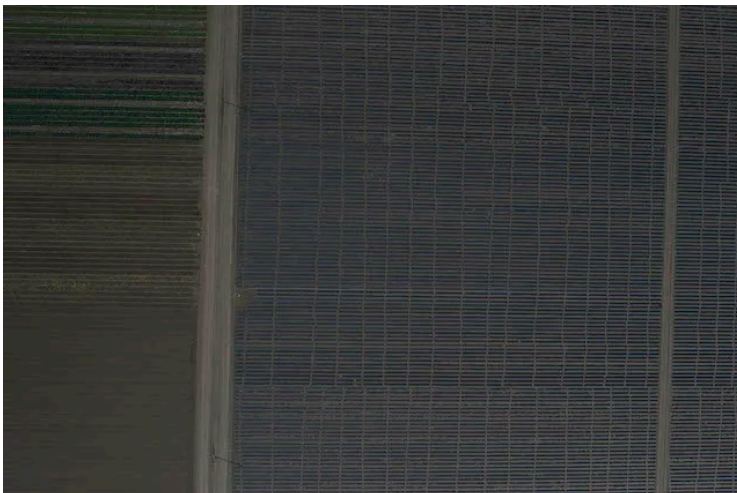
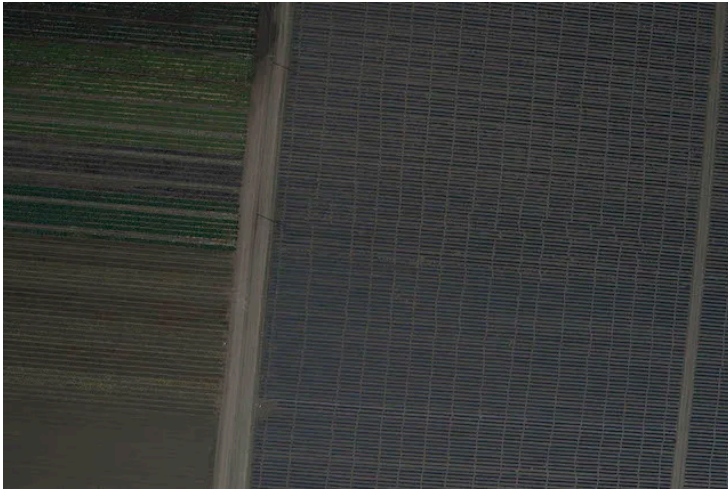


Aluno: Herich Gabriel de Campos

Relatório Prático: Visão Computacional - Geração de Imagens Panorâmicas

Imagens base utilizada para os panoramas:



1. Análise dos Resultados e Casos Extremos

Durante os testes de costura (Stitching) de imagens panorâmicas, foram testadas diferentes combinações de detectores (SIFT, ORB) e correspondentes (BF, FLANN) em cenários variados. Observou-se que o comportamento dos algoritmos muda drasticamente de acordo com as características geométricas e texturais das fotos originais.

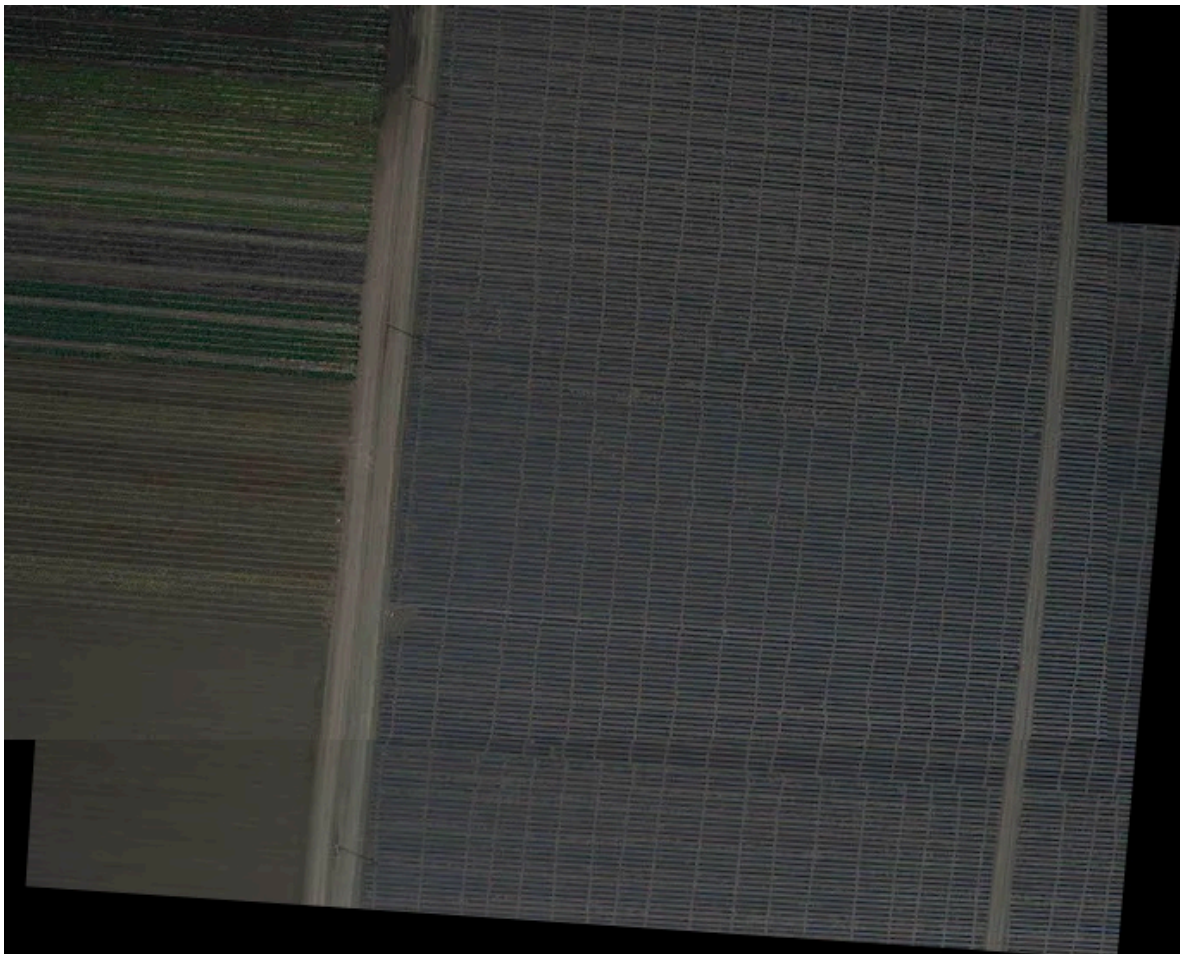
A) Imagens Aéreas com Movimento Diagonal (Imagens 1) Ao processar as imagens aéreas, o resultado apresentou a segunda imagem deformada dentro de um formato semelhante a um paralelogramo (ou trapézio inclinado).

- **Por que isso acontece:** A captura das imagens não ocorreu em um movimento puramente lateral (panorâmico horizontal). Houve mudança de perspectiva e rotação na captura por satélite/drone. O cálculo da matriz de **Homografia (Transformação de Perspectiva)** forçou a segunda imagem a sofrer rotação, inclinação e translação severas no plano cartesiano para que seus pixels coincidisse perfeitamente com a primeira imagem, resultando nessa geometria não retangular.



B) Cenário de Plantação Agrícola (Imagens 2) Este foi o cenário mais desafiador. Observaram-se os seguintes comportamentos:

1. **SIFT (com BF ou FLANN):** Gerou apenas "linhas esticadas" ou colapsou a imagem em um canto.
 2. **ORB + FLANN:** Apresentou resultados inconsistentes, alinhando em algumas execuções e falhando em outras.
 3. **ORB + BF:** Foi a única combinação capaz de realizar o alinhamento com estabilidade.
- **Por que isso acontece:** O problema central desta imagem é a **alta repetição de texturas (autossimilaridade)**. As linhas da plantação são visualmente idênticas.
 - O algoritmo **SIFT** utiliza o "Teste de Lowe" para descartar pontos ambíguos. Como as linhas da plantação são iguais, o SIFT encontra múltiplos candidatos válidos para o mesmo ponto e acaba descartando todos. Sem pontos suficientes (mínimo de 4), a matriz de Homografia sofre um colapso matemático, gerando distorções extremas.
 - O **FLANN** é um algoritmo probabilístico (baseado em aproximações). Em texturas repetitivas, essa aproximação frequentemente erra o "caminho" da busca.
 - O **ORB + BF (Brute-Force)** força o sistema a comparar matematicamente todos os pontos sem aproximações, conseguindo sobreviver à ambiguidade da textura e realizando a costura.



2. Análise de Custo-Benefício de Algoritmos (Requisito 4b)

Com base nos testes empíricos e na teoria de extração de características, seguem as recomendações de uso para diferentes aplicações do mundo real:

I. Aplicações de Baixo Consumo (Dispositivos Móveis, Robôs, Embarcados)

- **Algoritmo Recomendado:** ORB + BF (com distância de Hamming).
- **Explicação:** Sistemas embarcados ou alimentados por bateria possuem limitações severas de CPU. O algoritmo SIFT extrai descritores pesados baseados em ponto flutuante, exigindo cálculos complexos que drenam a bateria. O **ORB**, por outro lado, foi projetado para ser computacionalmente barato, extraindo descritores **binários** (sequências de 0s e 1s). Quando combinado com o BF utilizando a *Distância de Hamming*, a comparação de pontos é feita em nível de hardware por meio de operações lógicas bit a bit (XOR), o que é extremamente rápido e consome pouquíssima energia.

II. Grandes Conjuntos de Dados (Big Data)

- **Algoritmo Recomendado:** SIFT ou ORB combinados estritamente com **FLANN**.
- **Explicação:** Em Big Data (milhões de imagens), o gargalo computacional não é encontrar os pontos, mas sim combiná-los (Matching). O método Brute-Force (BF) compara cada ponto de uma imagem com todos os pontos da outra imagem, resultando em uma complexidade de tempo quadrática $O(N^2)$. Para processamento em larga escala, isso é inviável. O **FLANN** constrói árvores de busca (KD-Trees para ponto flutuante ou LSH para binários) que encontram os "vizinhos mais próximos de forma aproximada", reduzindo a complexidade de busca para escala logarítmica e viabilizando o processamento em massa.

III. Alta Qualidade e Fotos Profissionais

- **Algoritmo Recomendado:** SIFT + BF.
- **Explicação:** Quando a qualidade e a precisão do alinhamento são a prioridade máxima e não há restrições rígidas de tempo ou CPU, o **SIFT** é superior. Diferente do ORB, o SIFT captura com altíssima riqueza de detalhes a geometria local em vetores de ponto flutuante. Ele possui excelente invariância a mudanças bruscas de escala, iluminação e rotação. O uso do **BF** garante que não haverá aproximações probabilísticas na costura, garantindo a correspondência matematicamente mais exata possível, evitando artefatos visuais ou "fantasmas" na sobreposição das fotografias.

3. Implementação da Interface Gestual (Requisito 3b)

A funcionalidade de controle gestual para apresentação de slides foi implementada utilizando a técnica de **Fluxo Óptico Esparsa de Lucas-Kanade** (`cv2.calcOpticalFlowPyrLK`), em conjunto com a detecção de pontos de interesse de Shi-Tomasi (`cv2.goodFeaturesToTrack`). A automação dos comandos de teclado foi realizada pela biblioteca `pyautogui`.

A. Lógica de Funcionamento: O pipeline de rastreamento segue a seguinte estrutura:

1. **Inicialização e ROI:** Para evitar o rastreamento de ruídos de fundo (como pessoas passando ou objetos em movimento na cena), o sistema inicialmente define uma Região de Interesse (ROI) central, orientando o usuário a posicionar a mão nesta área.
2. **Extração de Características:** A função `goodFeaturesToTrack` analisa a ROI para encontrar pontos "fortes" na mão do usuário (ex: cantos ou regiões de alto contraste) que servirão como referência.
3. **Rastreamento (Tracking):** A cada novo frame capturado pela câmera, o método de Lucas-Kanade compara o frame atual com o anterior para determinar o vetor de deslocamento (Fluxo Óptico) do ponto rastreado.
4. **Tradução para Comandos:** O vetor de deslocamento horizontal (dx) é calculado.
 - Se dx for superior a um limiar positivo (ex: +60 pixels), o sistema interpreta como um gesto rápido para a direita e aciona a tecla correspondente para avançar o slide.
 - Se dx for inferior a um limiar negativo (ex: -60 pixels), o gesto é interpretado para a esquerda (retroceder slide).
5. **Controle de Fluxo (Cooldown):** Para evitar disparos múltiplos indesejados originados de um único movimento longo, implementou-se um mecanismo de *cooldown* (pausa), suspendendo a leitura de novos gestos por um número pré-definido de frames após cada comando executado. Após esse período, o rastreamento é reiniciado.

B. Desafios e Observações: Durante os testes, observou-se que o método de Lucas-Kanade puro apresenta certa sensibilidade a fatores externos:

- **Iluminação e Contraste:** A perda do ponto de rastreio (feature tracking loss) pode ocorrer se a mão do usuário não apresentar contraste suficiente com o fundo ou em condições de baixa luminosidade.
- **Velocidade do Movimento:** Movimentos excessivamente bruscos podem quebrar a premissa de pequenos deslocamentos assumida pelo algoritmo de Lucas-Kanade, exigindo a reinicialização da busca pelos pontos (`goodFeaturesToTrack`).

Apesar dessas sensibilidades inerentes ao algoritmo, com a calibração adequada dos limiares de deslocamento (dx) e do *cooldown*, o sistema atendeu satisfatoriamente ao requisito de navegação gestual em apresentações.

Código fonte

O trabalho foi feito utilizando a seguinte estrutura de arquivos

trabalho_vc/

```
|
|— main.py          # Onde fica a sua interface Tkinter
|— funcoes/         # Pasta com os módulos de processamento
|   |— __init__.py  # Arquivo vazio para o Python reconhecer a pasta
|   |— panorama.py  # Código do SIFT/ORB e Homografia
|   |— gestos.py     # Código do fluxo óptico (Lucas-Kanade)
|— resultados/      # Pasta onde ficam salvas as imagens de resultado
```

main.py

```
import cv2
import numpy as np
from tkinter import *
from tkinter import filedialog, ttk, messagebox
from PIL import Image, ImageTk
import os, datetime

# Importa as novas funções
from funcoes.panorama import gerar_panoramica
from funcoes.gestos import ControladorGestos

class AppVisaoComputacional:
    def __init__(self, master):
        self.master = master
        master.title("Trabalho Prático - Visão Computacional")
        master.geometry("1200x720")
        master.configure(bg="#222")

        # Variáveis da Panorâmica
        self.caminho_img1 = None
        self.caminho_img2 = None
        self.img_resultado = None

        # Variáveis de Vídeo
        self.cap = None
        self.feed_active = False
        self.controlador_gestos = ControladorGestos()

        self.construir_interface()
```

```

def construir_interface(self):
    main_frame = Frame(self.master, bg="#222")
    main_frame.pack(fill=BOTH, expand=True)

    # Painei Esquerdo (Controles)
    left_panel = Frame(main_frame, width=320, bg="#222")
    left_panel.pack(side=LEFT, fill=Y, padx=10, pady=10)
    left_panel.pack_propagate(False)

    # Abas
    notebook = ttk.Notebook(left_panel)
    notebook.pack(fill=BOTH, expand=True)

    self.tab_panorama = Frame(notebook, bg="#111")
    self.tab_gestos = Frame(notebook, bg="#111")

    notebook.add(self.tab_panorama, text="Panorâmica (Req
3a) ")
    notebook.add(self.tab_gestos, text="Gestos (Req 3b)")

    self.construir_aba_panorama()
    self.construir_aba_gestos()

    # Painei Direito (Exibição de Imagens/Vídeo)
    right_panel = Frame(main_frame, bg="#333")
    right_panel.pack(side=LEFT, fill=BOTH, expand=True,
padx=10, pady=10)

    self.label_tempo = Label(right_panel, text="Tempo de
Processamento: -- s", bg="#333", fg="#0f0", font=("Arial", 12,
"bold"))
    self.label_tempo.pack(pady=5)

    self.canvas_exibicao = Label(right_panel, bg="#000")
    self.canvas_exibicao.pack(expand=True, fill=BOTH,
padx=5, pady=5)

    def construir_aba_panorama(self):
        f = self.tab_panorama

        # Botões para carregar imagens
        Label(f, text="Imagens de Entrada", bg="#111",
fg="#fff", font=("Arial", 12, "bold")).pack(pady=10)

```

```

        self.lbl_img1 = Label(f, text="Imagem 1: Nenhuma",
bg="#111", fg="#ccc")
        self.lbl_img1.pack()
        Button(f, text="Carregar Imagem 1", width=25,
command=lambda: self.carregar_imagem(1)).pack(pady=5)

        self.lbl_img2 = Label(f, text="Imagem 2: Nenhuma",
bg="#111", fg="#ccc")
        self.lbl_img2.pack()
        Button(f, text="Carregar Imagem 2", width=25,
command=lambda: self.carregar_imagem(2)).pack(pady=5)

        ttk.Separator(f, orient=HORIZONTAL).pack(fill=X,
pady=15)

        # Seleção de Algoritmos
        Label(f, text="Configuração de Algoritmos", bg="#111",
fg="#fff", font=("Arial", 12, "bold")).pack(pady=5)

        Label(f, text="Encontrar Pontos (Detector):",
bg="#111", fg="#fff").pack()
        self.cb_detector = ttk.Combobox(f, values=["ORB",
"SIFT"], state="readonly")
        self.cb_detector.current(0)
        self.cb_detector.pack(pady=5)

        Label(f, text="Correspondência (Matcher):", bg="#111",
fg="#fff").pack()
        self.cb_matcher = ttk.Combobox(f, values=["BF",
"FLANN"], state="readonly")
        self.cb_matcher.current(0)
        self.cb_matcher.pack(pady=5)

        ttk.Separator(f, orient=HORIZONTAL).pack(fill=X,
pady=15)

        Button(f, text="⚙️ GERAR PANORÂMICA", width=25,
bg="#007bff", fg="white", font=("Arial", 10, "bold"),
command=self.executar_panorama).pack(pady=10)
        Button(f, text="💾 Salvar Resultado", width=25,
command=self.salvar_resultado).pack(pady=5)

    def construir_aba_gestos(self):

```



```

        f = self.tab_gestos
        Label(f, text="Controle de Slides", bg="#111",
fg="#fff", font=("Arial", 12, "bold")).pack(pady=10)

        Label(f, text="1. Abra uma apresentação (ex:
PowerPoint)\n2. Ative a câmera abaixo\n3. Coloque a mão no
quadrado azul\n4. Mova rápido para Direita/Esquerda",
bg="#111", fg="#ccc", justify=LEFT).pack(pady=10)

        Button(f, text="Iniciar Câmera / Gestos", width=25,
bg="#28a745", fg="white",
command=self.iniciar_camera).pack(pady=10)
        Button(f, text="Parar Câmera", width=25, bg="#dc3545",
fg="white", command=self.parar_camera).pack(pady=5)

        # --- Lógica Panorâmica ---
        def carregar_imagem(self, num):
            self.parar_camera()

            filepath =
filedialog.askopenfilename(filetypes=[("Imagens", "*.png *.jpg
*.jpeg")])
            if filepath:
                if num == 1:
                    self.caminho_img1 = filepath
                    self.lbl_img1.config(text=f"Imagem 1:
{os.path.basename(filepath)}")
                else:
                    self.caminho_img2 = filepath
                    self.lbl_img2.config(text=f"Imagem 2:
{os.path.basename(filepath)}")
                    messagebox.showinfo("Sucesso", f"Imagem {num}
carregada com sucesso!")

        def executar_panorama(self):
            if not self.caminho_img1 or not self.caminho_img2:
                messagebox.showwarning("Aviso", "Carregue as DUAS
imagens primeiro!")
            return

        self.parar_camera()
        det = self.cb_detector.get()
        mat = self.cb_matcher.get()

        try:

```

```

                                resultado, tempo =
gerar_panoramica(self.caminho_img1, self.caminho_img2, det,
mat)

        self.img_resultado = resultado

        # Atualiza UI
        self.label_tempo.config(text=f"Tempo de
Processamento ({det}+{mat}): {tempo:.4f} s")
        self.exibir_imagem(resultado)

    except Exception as e:
        messagebox.showerror("Erro na Panorâmica", str(e))

    def salvar_resultado(self):
        if self.img_resultado is None:
            messagebox.showwarning("Aviso", "Nenhum resultado
para salvar!")
            return
        os.makedirs("resultados", exist_ok=True)
        filename =
f"panorama_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}
.jpg"
        path = os.path.join("resultados", filename)
        cv2.imwrite(path, self.img_resultado)
        messagebox.showinfo("Salvo", f"Imagem salva
em:\n{path}")

    # --- Lógica de Câmera / Gestos ---
    def iniciar_camera(self):
        self.parar_camera()
        self.cap = cv2.VideoCapture(0)
        if not self.cap.isOpened():
            messagebox.showerror("Erro", "Não foi possível
acessar a webcam.")
            return

        self.feed_active = True
        self.label_tempo.config(text="Controle Gestual Ativo
(Lucas-Kanade)")
        self.atualizar_frame()

    def parar_camera(self):
        self.feed_active = False
        if self.cap:

```

```

        self.cap.release()
        self.cap = None

    def atualizar_frame(self):
        if not self.feed_active or not self.cap:
            return

        ret, frame = self.cap.read()
        if ret:
            # Passa o frame para o controlador de gestos
            frame_processado = self.controlador_gestos.processar_frame(frame)
            self.exibir_imagem(frame_processado)

        # Loop do Tkinter
        self.master.after(30, self.atualizar_frame)

    # --- Utilitário de Exibição ---
    def exibir_imagem(self, img_cv):
        try:
            # Converte BGR para RGB para o PIL
            img_rgb = cv2.cvtColor(img_cv, cv2.COLOR_BGR2RGB)

            # Redimensiona mantendo proporção para caber na
            h_tela = self.canvas_exibicao.wininfo_height() or
            600
            w_tela = self.canvas_exibicao.wininfo_width() or 800

            h_img, w_img = img_rgb.shape[:2]
            ratio = min(w_tela / w_img, h_tela / h_img)
            new_size = (int(w_img * ratio), int(h_img *
ratio))

            img_resized = cv2.resize(img_rgb, new_size,
interpolation=cv2.INTER_AREA)

            # Atualiza Canvas
            im_pil = Image.fromarray(img_resized)
            self.tk_img = ImageTk.PhotoImage(image=im_pil)
            self.canvas_exibicao.config(image=self.tk_img)
            self.canvas_exibicao.image = self.tk_img
        except Exception as e:

```

```

        print("Erro ao exibir imagem:", e)

if __name__ == "__main__":
    root = Tk()
    app = AppVisaoComputacional(root)
    root.mainloop()

```

panorama.py

```

import cv2
import numpy as np
import time

def gerar_panoramica(caminho_img1, caminho_img2,
detector_tipo, matcher_tipo):
    inicio = time.time()

    img1 = cv2.imread(caminho_img1)
    img2 = cv2.imread(caminho_img2)

    if img1 is None or img2 is None:
        raise ValueError("Erro ao carregar as imagens.")

    gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

    if detector_tipo == 'ORB':
        detector = cv2.ORB_create()
    elif detector_tipo == 'SIFT':
        detector = cv2.SIFT_create()
    else:
        raise ValueError("Detector inválido.")

    kp1, des1 = detector.detectAndCompute(gray1, None)
    kp2, des2 = detector.detectAndCompute(gray2, None)

    if des1 is None or des2 is None:
        raise ValueError("Não foi possível extrair
descritores.")

    if detector_tipo == 'SIFT':

```

```

des1 = np.float32(des1)
des2 = np.float32(des2)

good_matches = []

if matcher_tipo == 'BF':
    if detector_tipo == 'ORB':
        bf = cv2.BFMatcher(cv2.NORM_HAMMING,
crossCheck=True)
        matches = bf.match(des1, des2)
        good_matches = sorted(matches, key=lambda x:
x.distance)[:50]
    else:
        bf = cv2.BFMatcher()
        matches = bf.knnMatch(des1, des2, k=2)
        for m, n in matches:
            if m.distance < 0.75 * n.distance:
                good_matches.append(m)

elif matcher_tipo == 'FLANN':
    if detector_tipo == 'ORB':
        index_params = dict(algorithm=6, table_number=6,
key_size=12, multi_probe_level=1)
        search_params = dict(checks=50)
    else:
        index_params = dict(algorithm=1, trees=5)
        search_params = dict(checks=50)

        flann = cv2.FlannBasedMatcher(index_params,
search_params)
        matches = flann.knnMatch(des1, des2, k=2)

        for match in matches:
            if len(match) == 2:
                m, n = match
                if m.distance < 0.7 * n.distance:
                    good_matches.append(m)

if len(good_matches) >= 4:
    src_pts = np.float32([kp2[m.trainIdx].pt for m in
good_matches]).reshape(-1, 1, 2)
    dst_pts = np.float32([kp1[m.queryIdx].pt for m in
good_matches]).reshape(-1, 1, 2)

```



```

        H, _ = cv2.findHomography(src_pts, dst_pts,
cv2.RANSAC, 5.0)

        h1, w1 = img1.shape[:2]
        h2, w2 = img2.shape[:2]

        cantos_img1 = np.float32([[0, 0], [0, h1], [w1, h1],
[w1, 0]]).reshape(-1, 1, 2)
        cantos_img2 = np.float32([[0, 0], [0, h2], [w2, h2],
[w2, 0]]).reshape(-1, 1, 2)

        cantos_img2_transformados =
cv2.perspectiveTransform(cantos_img2, H)

        todos_cantos = np.concatenate((cantos_img1,
cantos_img2_transformados), axis=0)

        [xmin,      ymin]      =
np.int32(todos_cantos.min(axis=0).ravel() - 0.5)
        [xmax,      ymax]      =
np.int32(todos_cantos.max(axis=0).ravel() + 0.5)

        t = [-xmin, -ymin]
        Ht = np.array([[1, 0, t[0]], [0, 1, t[1]], [0, 0, 1]])

        resultado = cv2.warpPerspective(img2, Ht.dot(H), (xmax
- xmin, ymax - ymin))

        resultado[t[1]:h1+t[1], t[0]:w1+t[0]] = img1

        gray_res = cv2.cvtColor(resultado, cv2.COLOR_BGR2GRAY)
        _, thresh = cv2.threshold(gray_res, 1, 255,
cv2.THRESH_BINARY)
        contornos, _ = cv2.findContours(thresh,
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

        if contornos:
            x, y, w, h = cv2.boundingRect(contornos[0])
            resultado = resultado[y:y+h, x:x+w]

        fim = time.time()
        tempo_proc = fim - inicio

        return resultado, tempo_proc
    else:

```

```
        raise ValueError(f"Apenas {len(good_matches)}  
correspondências encontradas (mínimo 4).")
```

gestos.py

```
import cv2  
import numpy as np  
import pyautogui  
  
class ControladorGestos:  
    def __init__(self):  
        # Parâmetros do Lucas-Kanade  
        self.lk_params = dict(winSize=(15, 15),  
                               maxLevel=2,  
                               criteria=(cv2.TERM_CRITERIA_EPS  
| cv2.TERM_CRITERIA_COUNT, 10, 0.03))  
        # Parâmetros para achar os pontos iniciais da mão  
        self.feature_params = dict(maxCorners=1,  
qualityLevel=0.3, minDistance=7, blockSize=7)  
  
        self.old_gray = None  
        self.p0 = None  
        self.cooldown = 0 # Evita que passe 50 slides de uma  
vez  
  
    def processar_frame(self, frame):  
        # Inverte como um espelho para ficar intuitivo  
        frame = cv2.flip(frame, 1)  
        frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)  
  
        if self.cooldown > 0:  
            self.cooldown -= 1  
  
        # Se não tem ponto sendo rastreado, tenta encontrar um  
no centro da tela  
        if self.p0 is None:  
            altura, largura = frame_gray.shape  
            # Pega uma "caixa" no centro da tela para procurar  
a mão  
            roi = frame_gray[int(altura*0.2):int(altura*0.8),  
int(largura*0.2):int(largura*0.8)]
```

```

        pontos = cv2.goodFeaturesToTrack(roi, mask=None,
**self.feature_params)

        if pontos is not None:
            # Ajusta as coordenadas da ROI para a tela
inteira

            for p in pontos:
                p[0][0] += int(largura*0.2)
                p[0][1] += int(altura*0.2)
            self.p0 = pontos
            self.old_gray = frame_gray.copy()

            # Desenha um retângulo mostrando onde colocar a
mão

                cv2.rectangle(frame, (int(largura*0.2),
int(altura*0.2)), (int(largura*0.8), int(altura*0.8)), (255,
0, 0), 2)

                cv2.putText(frame, "Coloque a mao no quadrado",
(10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255,0,0), 2)
            return frame

        # Calcula o fluxo óptico
        p1, st, _ = cv2.calcOpticalFlowPyrLK(self.old_gray,
frame_gray, self.p0, None, **self.lk_params)

        # Se encontrou o ponto no novo frame
        if p1 is not None and st[0][0] == 1:
            novo_ponto = p1[0][0]
            ponto_antigo = self.p0[0][0]

            # Calcula a diferença no eixo X
            dx = novo_ponto[0] - ponto_antigo[0]

            if self.cooldown == 0:
                if dx > 60: # Gesto rápido para a direita
                    pyautogui.press('right')
                    print("Slide -> Avançar")
                    self.cooldown = 30 # Pausa os cálculos por
30 frames

                    self.p0 = None # Reseta o ponto para achar
a mão parada de novo
                elif dx < -60: # Gesto rápido para a esquerda
                    pyautogui.press('left')
                    print("Slide <- Retroceder")

```

```

        self.cooldown = 30
        self.p0 = None
    else:
        self.p0 = p1[st == 1].reshape(-1, 1, 2)
    else:
        self.p0 = p1[st == 1].reshape(-1, 1, 2)
        cv2.putText(frame, "Aguarde...", (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,0,255), 2)

        # Desenha uma bolinha verde onde está rastreando
        cv2.circle(frame, (int(novo_ponto[0]),
int(novo_ponto[1])), 10, (0, 255, 0), -1)
    else:
        self.p0 = None # Perdeu o rastreamento, vai procurar
de novo

    self.old_gray = frame_gray.copy()
    return frame

```